

Bat Monitoring Site Analysis

Jialun Darren Huang

Setup

```
library(readr)
library(dplyr)
library(knitr)
library(stringr)
library(lubridate)
library(MVN)
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.5.3
```

The library packages are as follows

- readr: Used to read csv files
- dplyr: Used to manipulate dataset
- knitr: Used to display dataframes in formatted tables
- stringr: Used to manipulate strings
- lubridate: Used to convert and manipulate date and time
- MVN: Used to find multivariate outliers
- forecast: Used to manipulate values using Box-Cox Transformation

Executive Summary

In this project, we look into how we can use preprocessing techniques to improve the data quality and integrity of a dataset. We will also look into merging dataset techniques as we use two datasets that have similar variables and are related to each other.

The procedure is as follows:

- Reading the Datasets
 - We will be importing the datasets into R
- Subsetting Observations
 - Instead of using the full dataset, we will take a sample by subsetting the number of observations
- Merging Datasets
 - Combining datasets with similar variable names that have similar values
- Tidy Dataset
 - After the dataset is merged, we can change the structure from an untidy format (wide) to tidy format (long)
- Variable Creation/Mutation
 - Create new variables from original variables
- Understand the Dataset
 - Once we have merged and tidy the dataset, we can take a closer look at the structure, convert variable data types and relabelling variables where necessary to improve the dataset quality
- Scanning Issues in Dataset
 - Identifying missing, special and inconsistent values to fix data integrity
 - Identifying univariate and multivariate outliers and either impute or remove them
- Data Transformation
 - Applied transformation methods like natural log on a specific variable to reduce skewness.

Finally, we discuss any challenges faced, ease and strengths of what we found easy to do, and what we have learnt from this whole project.

Data Description

In this project, we will be looking at two datasets that monitor and record temperature and humidity from bat monitoring sites.

Dataset 1 Link: Temperature and Humidity (Rising HF sensors) CSV (<https://discover.data.vic.gov.au/dataset/temperature-and-humidity-rising-hf-sensors/resource/14200bdb-fb16-51ba-a312-afac7f89a3d5>)

Dataset 2 Link: Temperature and Humidity for Bats monitoring CSV (<https://discover.data.vic.gov.au/dataset/temperature-and-humidity-for-bats-monitoring/resource/c501ce88-bf15-5e6a-b41a-3e1a145091a3>)

Read/Import Data

```
rhf_df = read_csv("rising-hf.csv")
bats_df = read_csv("temperature-and-humidity-monitoring-for-bats.csv")
kable(head(rhf_df[, 1:8], n = 3))
```

metadata_time	name	dev_id	location	payload_fields_temp	payload_fields_hum	payload_fields_battery	payload_fields_period
2025-02-14 10:11:00	Herbert Street, Belmont	risinghf- 8f9	-38.1777595, 144.3509038	12.98	99.96	3.21	900 sec
2025-02-14 10:04:14	NA	risinghf- 6aa	NA	20.64	60.89	3.32	900 sec
2025-02-14 15:19:50	Jeffrey Street, Indented Head	risinghf- 96f	-38.13978826, 144.7099685	12.81	87.75	3.24	900 sec

```
kable(head(rhf_df[, 9:length(colnames(rhf_df))], n = 3))
```

skip_record_flag	temperature	humidity
N	12.98	99.96
N	20.64	60.89
N	12.81	87.75

```
kable(head(bats_df, n = 3))
```

metadata_time	dev_id	name	payload_fields_battery	payload_fields_hum	payload_fields_temp	payload_fields_period	location
2025-02-14 15:06:44	risinghf- e1c	BAT- 3	3.27	87.75	13.08	900 sec	-38.147222, 144.384444
2025-02-14 10:17:28	risinghf- d58	BAT- 2	3.27	72.61	16.38	900 sec	-38.148056, 144.383611
2025-02-14 15:27:07	risinghf- e6b	BAT- 1	3.08	86.29	12.63	900 sec	-38.148611, 144.384167

We can briefly visualise the variables and a sample observation from each dataset. We will look more at the variables and structure later.

Subset Observations

We will subset both datasets to 80% of the data and set the random seed. This ensures that the results from the following code below can be reproduced and consistent.

```
set.seed(3755729)
```

We create a function `subset_df()`, to subset the dataset. You can control the percentage of subsetting data with `subset_perc`, with the default value at 80%.

```
subset_df <- function(df, subset_perc = 0.8) {
  rows <- ceiling(subset_perc * nrow(df))
  subset_df <- df[sample(nrow(df), rows), ]
  return(subset_df)
}
```

```
subset_rhf_df <- subset_df(rhf_df)
cat("Dim: ", dim(subset_rhf_df))
```

```
## Dim: 722432 11
```

```
subset_bats_df <- subset_df(bats_df)
cat("Dim: ", dim(subset_bats_df))
```

```
## Dim: 435502 8
```

Even after subsetting the datasets, the dataset is very large. We can subset it further. Lets further subset it to 1% from the subsetting dataset.

```
subset_rhf_df <- subset_df(subset_rhf_df, subset_perc = 0.01)
cat("Dim: ", dim(subset_rhf_df))
```

```
## Dim: 7225 11
```

```
subset_bats_df <- subset_df(subset_bats_df, subset_perc = 0.01)
cat("Dim: ", dim(subset_bats_df))
```

```
## Dim: 4356 8
```

We now have a subsetted dataset with 7225 observations and 11 variables, and another subsetted dataset with 4356 observations and 8 variables.

Merge Datasets

Next, we can merge the datasets together, using `bind_rows()`.

```
subset_merged_df <- bind_rows(subset_rhf_df, subset_bats_df)
```

Tidy & Manipulate Data I (Tidy Dataset)

Tidy principles - Each variable must have its own column. - Each observation must have its own row. - Each value must have its own cell.

Both datasets are untidy, as the `location` variable contains more than one value. In order for the dataset to be tidy, we have to separate the values to `lat` and `long`.

`extract_location_values()` function extracts the `location` values to `lat` and `long`. Note that there can be missing values in `location`. For now we will set both `lat` and `long` to also NA. We will deal with the missing values later.

```
extract_location_values <- function(loc_val) {
  if (is.na(loc_val)) {
    return(c(NA_real_, NA_real_))
  }
  location <- strsplit(loc_val, ",")[[1]]
  lat <- as.numeric(str_trim(location[1]))
  long <- as.numeric(str_trim(location[2]))
  return(c(lat, long))
}
```

`convert_location_lat_long()` function uses the matrix array output from `extract_location_values()` and populates the dataframe with `lat` and `long` variables

```
convert_location_lat_long <- function(df) {
  lat_long <- sapply(df$location, extract_location_values)
  colnames(lat_long) <- NULL
  df$lat <- lat_long[1, ]
  df$long <- lat_long[2, ]
  return(df)
}
```

```
subset_merged_df <- convert_location_lat_long(subset_merged_df)
kable(subset_merged_df[1:3, c("location", "lat", "long")])
```

location	lat	long
-37.886444, 144.269282	-37.88644	144.2693
-38.13170461, 144.3431925	-38.13170	144.3432
-38.123897, 144.641986	-38.12390	144.6420

We can see that the values in `location` variable has been split to `lat` and `long`, tidying the dataset.

Tidy & Manipulate Data II (Variable Creation/Mutation)

In this section, we can extract the `year` and `month` values from `metadata_time`. This is done so that we can focus on the year and month of the observations.

`metadt_to_ind_dt()` function takes `metadata_time` and creates the `year` and `month` values.

```
metadt_to_ind_dt <- function(df) {  
  df <- df %>%  
    mutate(  
      year = as.character(year(metadata_time)),  
      month = month(metadata_time, label = TRUE)  
    )  
  return(df)  
}
```

```
subset_merged_df <- metadt_to_ind_dt(subset_merged_df)  
kable(subset_merged_df[1:3, c("metadata_time", "year", "month")])
```

metadata_time	year	month
2023-08-02 01:14:36	2023	Aug
2023-01-10 14:26:31	2023	Jan
2023-06-30 11:10:34	2023	Jun

Subset dataset by variables

In this section, we conduct an additional step, by selecting only variables that we want to focus on for the project. Each variable will be described later.

```
subset_merged_df <- subset_merged_df %>% select(year, month, name, skip_record_flag, lat, long, payload_fields_temp, payload_fields_hum, payload_fields_battery)
```

Understand

I will be renaming the columns here so that it can fit within the outputs in the .pdf documentation

```
subset_merged_df <- rename(subset_merged_df, temp = payload_fields_temp, hum = payload_fields_hum, bat = payload_fields_battery)
```

```
head(subset_merged_df, n = 3)
```

year	month	name	skip_record_flag	lat	long	temp	hum	bat
<chr>	<ord>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
2023	Aug	Mount Anakie, Anakie	N	-37.88644	144.2693	13.41	74.08	3.03
2023	Jan	Church Street, Geelong West	N	-38.13170	144.3432	17.87	85.31	3.41
2023	Jun	Tower Road, Portarlington	N	-38.12390	144.6420	10.30	92.63	2.99
3 rows								

```
str(subset_merged_df)
```

```
## tibble [11,581 × 9] (S3: tbl_df/tbl/data.frame)  
## $ year      : chr [1:11581] "2023" "2023" "2023" "2024" ...  
## $ month     : Ord.factor w/ 12 levels "Jan"<"Feb"<"Mar"<...: 8 1 6 10 1 12 11 2 6 10 ...  
## $ name      : chr [1:11581] "Mount Anakie, Anakie" "Church Street, Geelong West" "Tower Road, Portarlington" "Spec  
tacle Way, Leopold" ...  
## $ skip_record_flag: chr [1:11581] "N" "N" "N" "N" ...  
## $ lat       : num [1:11581] -37.9 -38.1 -38.1 -38.2 -38.3 ...  
## $ long      : num [1:11581] 144 144 145 144 144 ...  
## $ temp      : num [1:11581] 13.41 17.87 10.3 12.14 9.99 ...  
## $ hum       : num [1:11581] 74.1 85.3 92.6 89.2 100 ...  
## $ bat       : num [1:11581] 3.03 3.41 2.99 3.3 3.28 3.44 2.97 3.06 3.29 3.15 ...
```

Feature	Data Type	Description
year	character	The year of the recorded observation
month	Factor (Ordinal)	The month of the recorded observation
name	character	The name of the location
skip_record_flag	character	A binary categorical variable for skip_record, where “Y” is True and “N” is False
lat	numerical	The latitude of the location

Feature	Data Type	Description
long	numerical	The longitude of the location
temp	numerical	The temperature recorded
hum	numerical	The humidity recorded
bat	numerical	The voltage of the device recorded

Data Type Conversions

Some of the variables can be converted to a more specific data types in R, which helps R interpret them better and improve the quality of analysis

Feature	Data Type	Updated Data Type	Reason
year	character	Factor (Ordinal)	year is an categorical variable that has an order (i.e. 2024 is before 2025)
name	character	Factor	name is a nominal categorical variable
skip_record_flag	character	Integer	Relabelled to binary integers to have clarity in analysis and modelling. Similar to One-Hot Encoding, by converting it to integers, this allows models to use numeric (Integer) binary values instead of character labels for the target variable in classification tasks.

Lets take a closer look at `name`

```
unique(subset_merged_df$name)
```

```
## [1] "Mount Anakie, Anakie"      "Church Street, Geelong West"
## [3] "Tower Road, Portarlington"  "Spectacle Way, Leopold"
## [5] "Challis Avenue, Breamlea"   "Tamala Avenue, Marshall"
## [7] "Herbert Street, Belmont"    "Swan Bay Road, Mannerim"
## [9] "Sophie Way, Armstrong Creek" "The Esplanade, Indented Head"
## [11] NA                          "Trevally Drive, Ocean Grove"
## [13] "The Esplanade, Portarlington" "Ewing Blyth Drive, Barwon Heads"
## [15] "Adella Court, Moolap"       "Jeffrey Street, Indented Head"
## [17] "Merton Grove, Manifold Heights" "Kagoola Court, Norlane"
## [19] "Turner Place, South Geelong" "Seaspray Court, Indented Head"
## [21] "Secluded Drive, Armstrong Creek" "South Street, Belmont"
## [23] "Gurley Court, Grovedale"     "BAT-4"
## [25] "BAT-2"                      "BAT-1"
## [27] "BAT-3"                      "BAT-5"
```

We can see that there are only a couple of distinct `name` values. This means that the `name` is a nominal categorical variable. We will have to convert this from character to factor.

```
subset_merged_df$name <- factor(subset_merged_df$name)
subset_merged_df$year <- factor(subset_merged_df$year, ordered = TRUE)
```

Labelling Factor Variable

We will be relabelling the values, where "Y" is 1 and "N" is 0.

```
subset_merged_df$skip_record_flag <- ifelse(subset_merged_df$skip_record_flag == "Y", 1L, 0L)
```

Final Results after Data Conversion and Labelling

```
str(subset_merged_df)
```

```
## tibble [11,581 × 9] (S3: tbl_df/tbl/data.frame)
## $ year      : Ord.factor w/ 5 levels "2021"<"2022"<...: 3 3 3 4 4 3 3 5 4 3 ...
## $ month     : Ord.factor w/ 12 levels "Jan"<"Feb"<"Mar"<...: 8 1 6 10 1 12 11 2 6 10 ...
## $ name      : Factor w/ 27 levels "Adella Court, Moolap",...: 15 8 25 20 7 22 15 25 11 15 ...
## $ skip_record_flag: int [1:11581] 0 0 0 0 0 0 0 0 0 0 ...
## $ lat       : num [1:11581] -37.9 -38.1 -38.1 -38.2 -38.3 ...
## $ long      : num [1:11581] 144 144 145 144 144 ...
## $ temp      : num [1:11581] 13.41 17.87 10.3 12.14 9.99 ...
## $ hum       : num [1:11581] 74.1 85.3 92.6 89.2 100 ...
## $ bat       : num [1:11581] 3.03 3.41 2.99 3.3 3.28 3.44 2.97 3.06 3.29 3.15 ...
```

Scan I (Addressing Missing, Special and Inconsistent Values)

In this section, we look at common issues found in datasets that have to be cleaned. These include

1. Missing Values: Dataset with missing values are recorded as NA or NaN, and can cause mathematical problems.
2. Special and Inconsistent values: Dataset with such values do not hold the right information, and can cause bias results (e.g. We have an human age value recorded at 1000, but it is impossible for someone to live that long)

Missing Values

```
colSums(is.na(subset_merged_df))
```

```
##          year          month          name skip_record_flag
##           0             0           968           4356
##          lat           long          temp           hum
##         968           968             0             0
##          bat
##           0
```

There are missing values in `name`, `skip_record_flag`, `lat` and `long`.

The reason why there are missing values in `skip_record_flag` is only one dataset has the `skip_record_flag` variable. During merging, the dataset without this variable would result in the observations having NA for this variable. In this example, we can fill in the missing values, by setting `skip_record_flag` to 0, as we can assume that it may have been ignored since it is not recorded in that dataset.

```
subset_merged_df$skip_record_flag[is.na(subset_merged_df$skip_record_flag)] <- 0
```

Lets check if the missing `lat` and `long` are the same observations

```
sum(is.na(subset_merged_df$lat) != is.na(subset_merged_df$long))
```

```
## [1] 0
```

All observations either have `lat` and `long` coordinates, or missing both `lat` and `long` coordinates, we can use either variable to analyse or compute the observations with missing values. In this case, we will use `lat`

```
cat("No. of Observations ", dim(subset_merged_df)[1], " No. of missing coordinates observations ", sum(is.na(subset_merged_df$lat)))
```

```
## No. of Observations 11581 No. of missing coordinates observations 968
```

```
cat("Percentage of dropped observations ", (sum(is.na(subset_merged_df$lat)) / dim(subset_merged_df)[1]) * 100)
```

```
## Percentage of dropped observations 8.358518
```

This means that we can drop all 968 observations, which would be 8.36% of the observations dropped, slightly above the acceptable range of 5%, but is still generally acceptable. One way to work around this is to add more observations from the original datasets, since our subsetted data is less than 1% of the whole dataset.

```
subset_merged_df <- subset_merged_df[!(is.na(subset_merged_df$lat)), ]
```

After fixing Missing Values

```
colSums(is.na(subset_merged_df))
```

```
##          year          month          name skip_record_flag
##           0             0             0             0
##          lat           long          temp           hum
##           0             0             0             0
##          bat
##           0
```

```
cat("Dim: ", dim(subset_merged_df))
```

```
## Dim: 10613 9
```

There are no more missing values, and we are now left with 10613 observations.

Special and Inconsistent Values

Numerical Variables

`get_min_max_values()` function is used to display the min and max values for a numerical variable. We will use this function to see the range of values for all numerical variables.

```
numerical_variables = c("lat", "long", "temp", "hum", "bat")
get_min_max_values <- function(nums) {
  return(str_c("min: ", min(nums), " max: ", max(nums)))
}
```

- lat should be a value between -90 to 90
- long should be a value between - 180 to 180
- hum should be a value between 0 to 100
- bat should be a value above 0, but not too extreme. Voltage should not be too high
- temp should be an acceptable range, it cannot be too low (-1000 Celsius is too low) or too hot (1000 Celsius is too high)

```
for(var in numerical_variables) {
  cat(var, get_min_max_values(subset_merged_df[[var]]), "\n")
}
```

```
## lat min: -38.29468833 max: -37.886444
## long min: 144.269282 max: 144.7099685
## temp min: -2.64 max: 47.02
## hum min: 20.86 max: 99.96
## bat min: 2.7 max: 3.52
```

All numerical variables are within acceptable range.

Categorical Variables

We will be looking for anomalies in the values (e.g. Incorrect spellings)

```
nominal_categorical_variables = c("name", "skip_record_flag")
ordinal_categorical_variables = c("year", "month")
```

```
for(var in nominal_categorical_variables) {
  cat(var, "\n", unique(as.character(subset_merged_df[[var]])), "\n")
}
```

```
## name
## Mount Anakie, Anakie Church Street, Geelong West Tower Road, Portarlington Spectacle Way, Leopold Challis Avenue, Breaml
ea Tamala Avenue, Marshall Herbert Street, Belmont Swan Bay Road, Mannerim Sophie Way, Armstrong Creek The Esplanade, Indent
ed Head Trevally Drive, Ocean Grove The Esplanade, Portarlington Ewing Blyth Drive, Barwon Heads Adella Court, Moolap Jeffre
y Street, Indented Head Merton Grove, Manifold Heights Kagoola Court, Norlane Turner Place, South Geelong Seaspray Court, In
dented Head Secluded Drive, Armstrong Creek South Street, Belmont Gurley Court, Grovedale BAT-4 BAT-2 BAT-1 BAT-3 BAT-5
## skip_record_flag
## 0
```

Some observations in `name` only contains the BAT location. We should check out if we can map the BAT locations to an actual location name for better clarity.

```
bat_loc <- c("BAT-1", "BAT-2", "BAT-3", "BAT-4", "BAT-5")
for(loc in bat_loc) {
  bat_lat_long <- unique(subset_merged_df[subset_merged_df$name == loc, c("lat", "long")])
  result <- unique(subset_merged_df[subset_merged_df$lat == bat_lat_long$lat &
    subset_merged_df$long == bat_lat_long$long, c("name")])
  cat("BAT locations found with the same coordinates as ", loc, ": ", as.character(result$name), "\n")
}
```

```
## BAT locations found with the same coordinates as BAT-1 : BAT-1
## BAT locations found with the same coordinates as BAT-2 : BAT-2
## BAT locations found with the same coordinates as BAT-3 : BAT-3
## BAT locations found with the same coordinates as BAT-4 : BAT-4
## BAT locations found with the same coordinates as BAT-5 : BAT-5
```

Unfortunately, there are no observations other than itself with the same `lat` and `long`, and cannot determine the actual location name. We will leave the BAT location names as it is for now, but we should replace the names with actual location names to improve readability and consistency throughout the dataset.

```
for(var in ordinal_categorical_variables) {
  cat(var, "\n", levels(subset_merged_df[[var]]), "\n")
}
```

```
## year
## 2021 2022 2023 2024 2025
## month
## Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

There are no issues with `year` and `month` variables.

Scan II (Addressing Outliers)

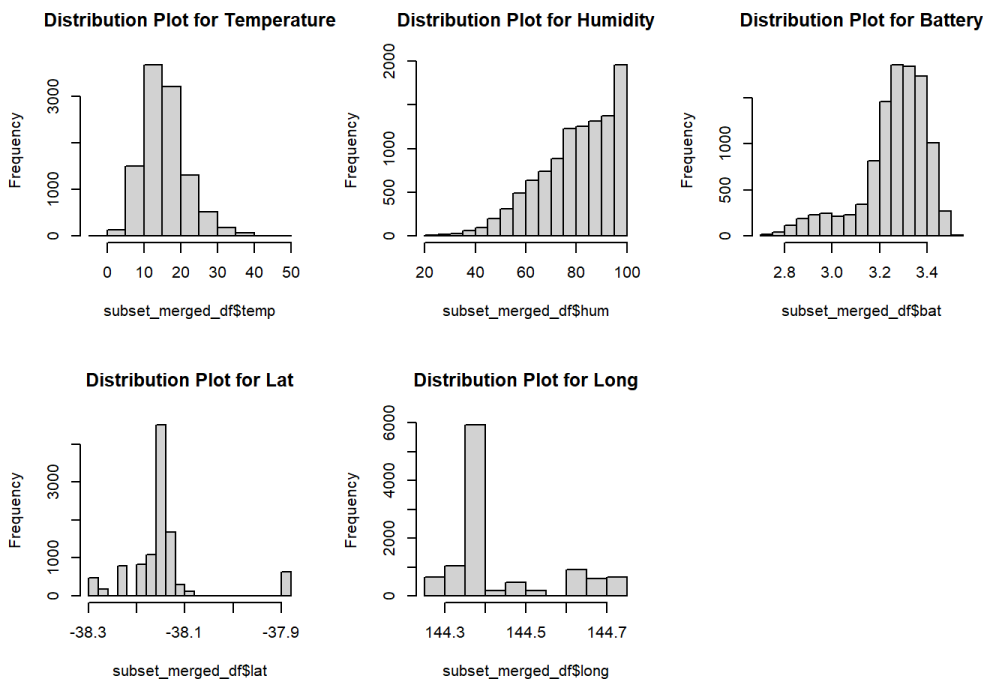
In this section, we will take a look at outliers that may appear in numeric variables. Identifying outliers are important as they can cause bias in statistical analysis.

We will be looking at both univariate outliers (outliers in a single variable) and multivariate outliers (outliers across multiple variables combined together)

Univariate Outliers

Lets take a look at the distribution plot for the variables

```
par(mfrow = c(2, 3))
hist(subset_merged_df$temp, main = "Distribution Plot for Temperature")
hist(subset_merged_df$hum, main = "Distribution Plot for Humidity")
hist(subset_merged_df$bat, main = "Distribution Plot for Battery")
hist(subset_merged_df$lat, main = "Distribution Plot for Lat")
hist(subset_merged_df$long, main = "Distribution Plot for Long")
```



We will also use Shapiro Test Statistic to check for normality in variables. Do note that we have to subset our dataset to conduct Shapiro Test in R as this works only for datasets with 3 to 5000 observations

```
for(num_var in numerical_variables) {
  cat(num_var, ": ", shapiro.test(subset_df(subset_merged_df, subset_perc = 0.04)[[num_var]])$p.value, "\n")
}
```

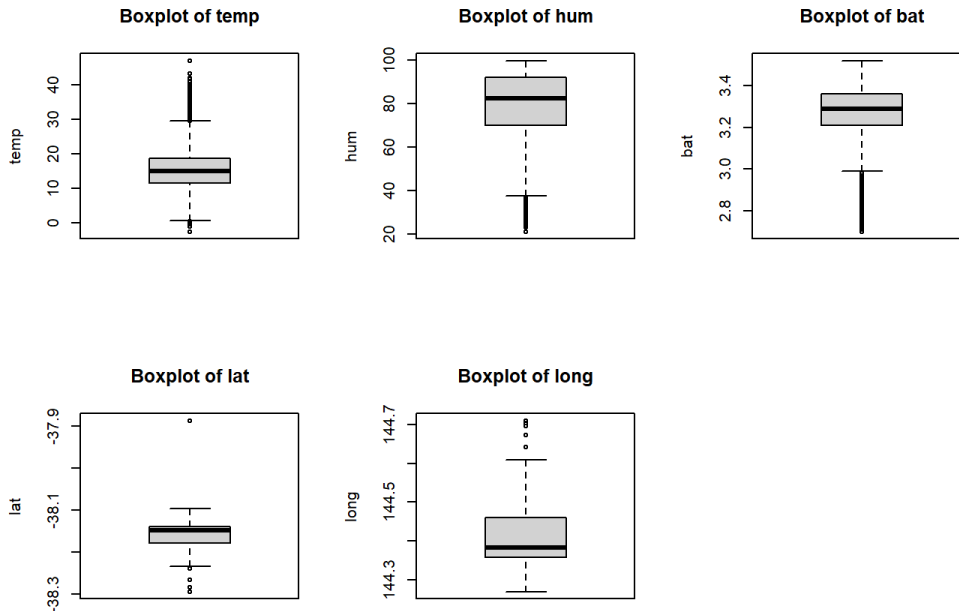
```
## lat : 2.970175e-26
## long : 2.142231e-23
## temp : 3.281934e-09
## hum : 4.714906e-11
## bat : 9.628863e-18
```

From the histograms, we can see that the distributions are skewed, and all p-values are less than 0.05, therefore the data is not normally distributed. For not normally distributed data, we can use boxplot to determine the outliers.

`display_boxplot_and_get_outlier_indices()` function displays the boxplot and returns the outlier indexes.

```
display_boxplot_and_get_outlier_indices <- function(var) {
  bp <- boxplot(subset_merged_df[[var]], main = str_c("Boxplot of ", var), ylab = var)
  outlier_indexes <- which(subset_merged_df[[var]] %in% bp$out)
  return(outlier_indexes)
}
```

```
par(mfrow = c(2, 3))
bp_temp <- display_boxplot_and_get_outlier_indices("temp")
bp_hum <- display_boxplot_and_get_outlier_indices("hum")
bp_bat <- display_boxplot_and_get_outlier_indices("bat")
bp_lat <- display_boxplot_and_get_outlier_indices("lat")
bp_long <- display_boxplot_and_get_outlier_indices("long")
```



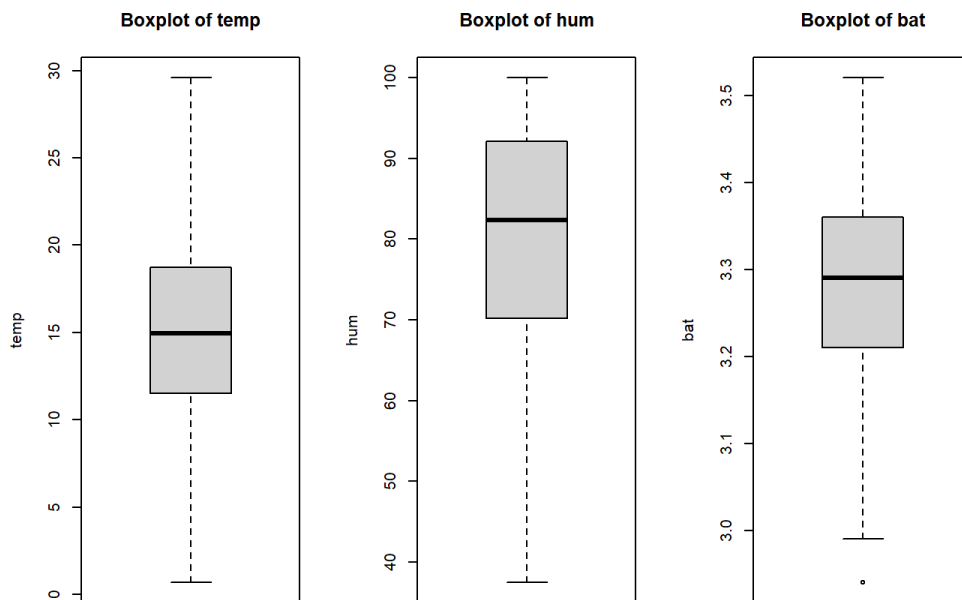
While we can drop outliers from the dataset, we can impute the outliers by capping. Capping replaces extreme outliers with a specified percentile value. In `univariate_cap()` function, we cap small value outliers to the 5th percentile and large value outliers to the 95th percentile. This allows us to keep the observations rather than dropping them.

```
univariate_cap <- function(x){
  quantiles <- quantile(x, c(.05, 0.25, 0.75, .95))
  x[x < quantiles[2] - 1.5 * IQR(x)] <- quantiles[1]
  x[x > quantiles[3] + 1.5 * IQR(x)] <- quantiles[4]
  return(x)
}
```

We will be capping `temp`, `hum` and `bat`. `lat` and `long` should not be capped since they specify the coordinates, and coordinates should not be changed, otherwise, you would be referring to a different location.

```
subset_merged_df$temp <- univariate_cap(subset_merged_df$temp)
subset_merged_df$hum <- univariate_cap(subset_merged_df$hum)
subset_merged_df$bat <- univariate_cap(subset_merged_df$bat)
```

```
par(mfrow = c(1, 3))
bp_temp <- display_boxplot_and_get_outlier_indices("temp")
bp_hum <- display_boxplot_and_get_outlier_indices("hum")
bp_bat <- display_boxplot_and_get_outlier_indices("bat")
```



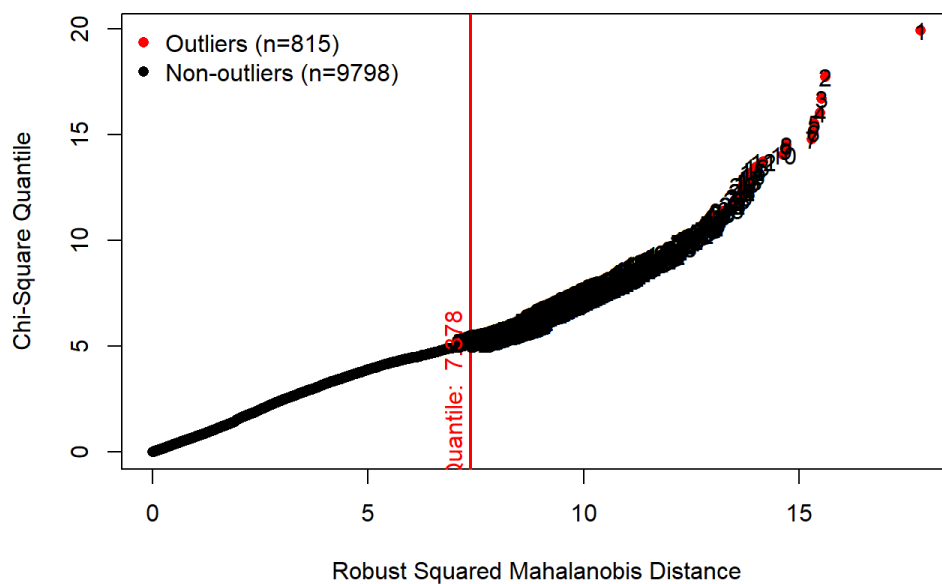
We can see that there are almost no more outliers for `temp`, `hum` and `bat`. It is possible to still detect some outliers in the variables, due to an updated set of quantile values.

Multivariate Outliers

We can use `mvn` function to find multivariate outliers. Lets do it for 4 different groups 1) `temp` and `hum` 2) `temp`, `hum` and `bat` 3) `lat` and `long` 4) All numeric variables

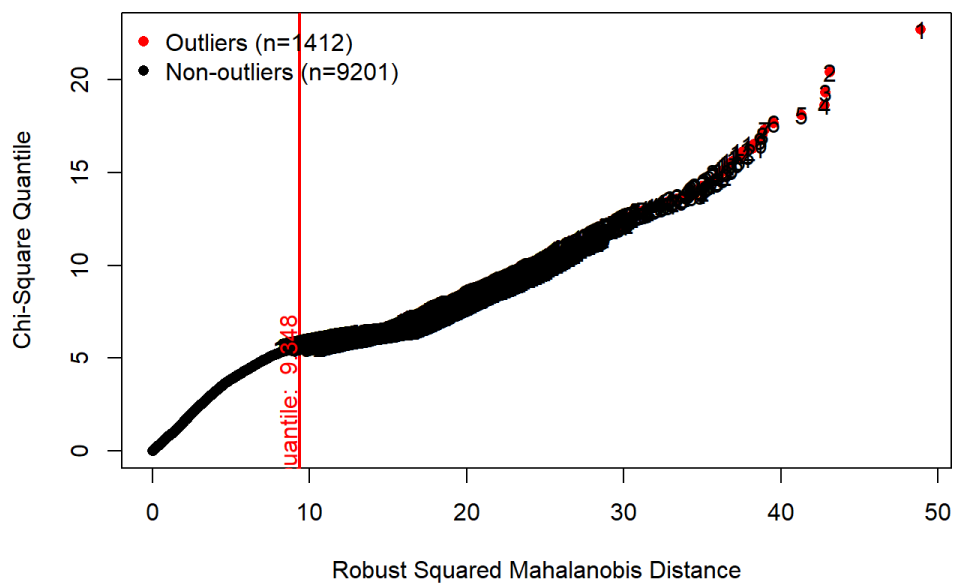
```
par(mfrow = c(1, 1))
mvn_temp_hum <- mvn(data = subset_merged_df[, c("temp", "hum")], multivariateOutlierMethod = "quan", showOutliers = TRUE)
```

Chi-Square Q-Q Plot



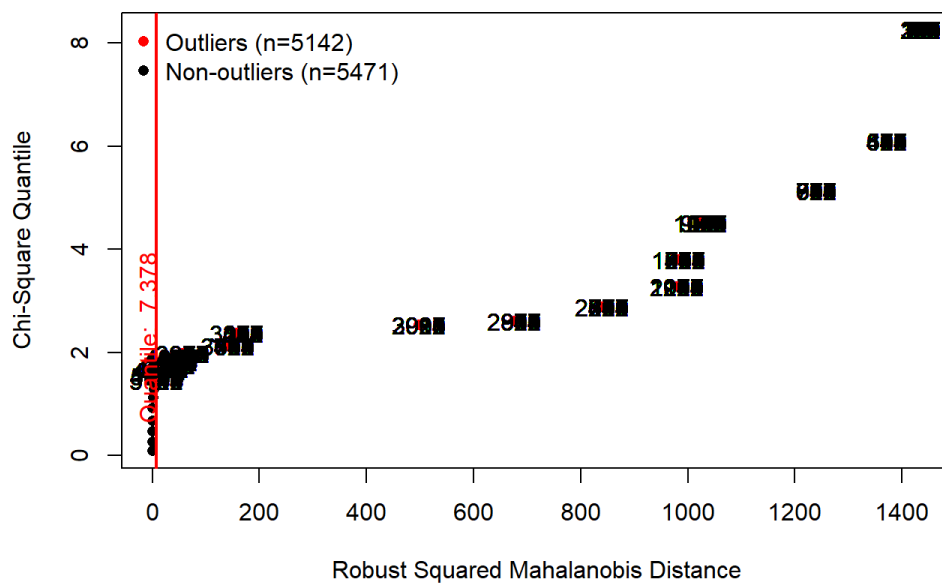
```
par(mfrow = c(1, 1))
mvn_temp_hum_bat <- mvn(data = subset_merged_df[, c("temp", "hum", "bat")],
  multivariateOutlierMethod = "quan", showOutliers = TRUE)
```

Chi-Square Q-Q Plot



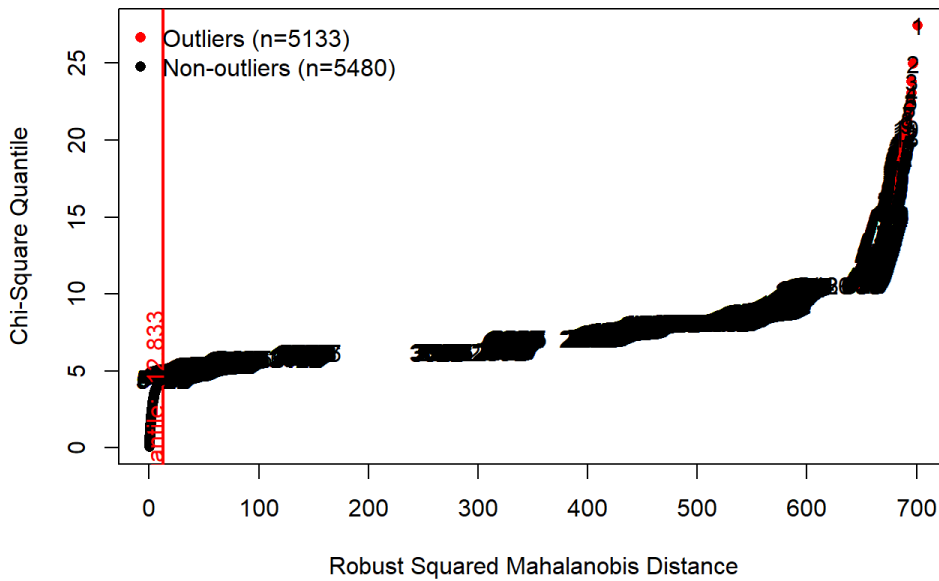
```
par(mfrow = c(1, 1))
mvn_lat_long <- mvn(data = subset_merged_df[, c("lat", "long")],
  multivariateOutlierMethod = "quan", showOutliers = TRUE)
```

Chi-Square Q-Q Plot



```
par(mfrow = c(1, 1))
mvn_all <- mvn(data = subset_merged_df[, c(numerical_variables)],
  multivariateOutlierMethod = "quan", showOutliers = TRUE)
```

Chi-Square Q-Q Plot



We can see that there are 815 outliers with `temp` and `hum`, and 1412 outliers with `temp`, `hum` and `bat` combined. Similar to univariate outliers, we should not be removing by `lat` and `long`. In this example, we will be dropping outliers detected for `temp`, `hum` and `bat`.

```
outlier_indexes <- which(mvn_temp_hum_bat$multivariateOutliers == TRUE)
subset_merged_df <- subset_merged_df[-outlier_indexes, ]
```

```
cat("Dim: ", dim(subset_merged_df))
```

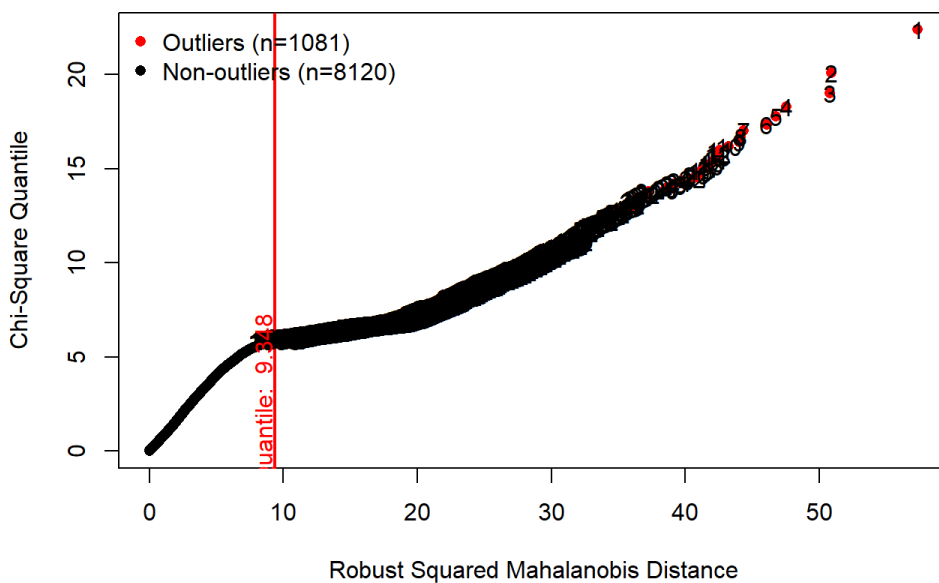
```
## Dim: 9201 9
```

After removing the outliers detected from `mvn()`, we are left with X observations.

Similar to removing univariate outliers, do note that it is possible to still detect multivariate outliers in the dataset, due to an updated set of quantile values.

```
par(mfrow = c(1, 1))
mvn_temp_hum_bat <- mvn(data = subset_merged_df[, c("temp", "hum", "bat")],
  multivariateOutlierMethod = "quan", showOutliers = TRUE)
```

Chi-Square Q-Q Plot



In this case, you can continue remove outliers by repeating the process, until all outliers are removed. Each process would reduce the number of detected outliers, however, affect the total number of observations, as you will be reducing it every time you run an iteration.

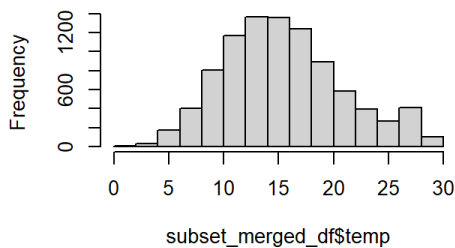
For the purpose of keeping as much data as possible, we will only remove the detected multivariate outliers once.

Transform (Data Transformation)

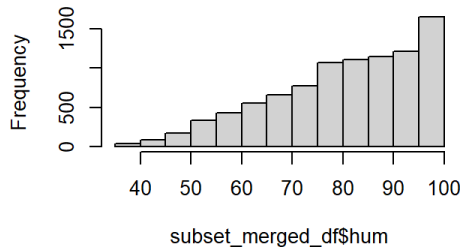
In this section, we will be applying data transformation to a variable. Lets see how this is done.

```
par(mfrow = c(2, 2))
hist(subset_merged_df$temp, main = "Distribution Plot for Temperature")
hist(subset_merged_df$hum, main = "Distribution Plot for Humidity")
hist(subset_merged_df$bat, main = "Distribution Plot for Battery")
```

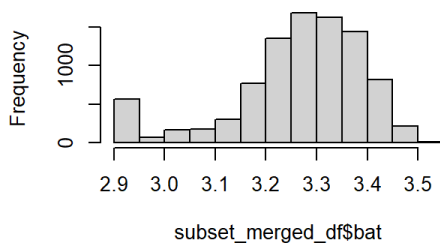
Distribution Plot for Temperature



Distribution Plot for Humidity



Distribution Plot for Battery

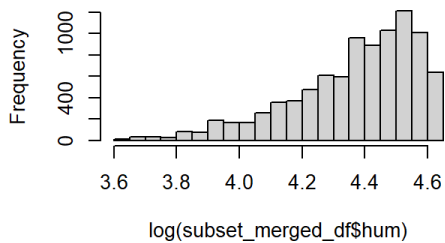


From the histograms, we can see that `hum` is left-skewed. We can conduct data transformation on `hum` to reduce the skewness. A more normally distributed variable can improve the analysis and results of statistical tests (e.g. ANOVA) and models that assume normality (e.g. Linear Regression).

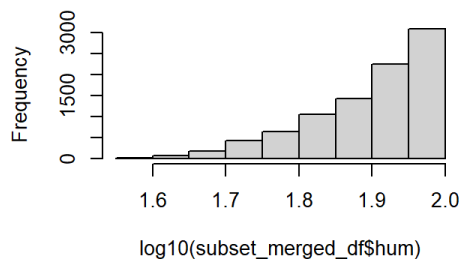
Lets look at some transformation methods 1) Natural Log `log()` - Applies natural log to the values 2) Log Base 10 `log10()` - Applies log base 10 to the values 3) Box-Cox Transfomration `BoxCox()` - Uses an value (lambda) to find the optimal result to normalise the distribution

```
par(mfrow = c(2, 2))
hist(log(subset_merged_df$hum), main = "Distribution Plot with natural log")
hist(log10(subset_merged_df$hum), main = "Distribution Plot with log base 10")
hist(BoxCox(subset_merged_df$hum, lambda = "auto"), main = "Distribution Plot with BoxCox")
```

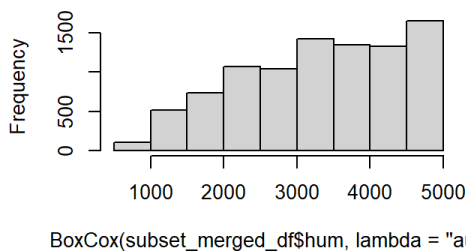
Distribution Plot with natural log



Distribution Plot with log base 10



Distribution Plot with BoxCox



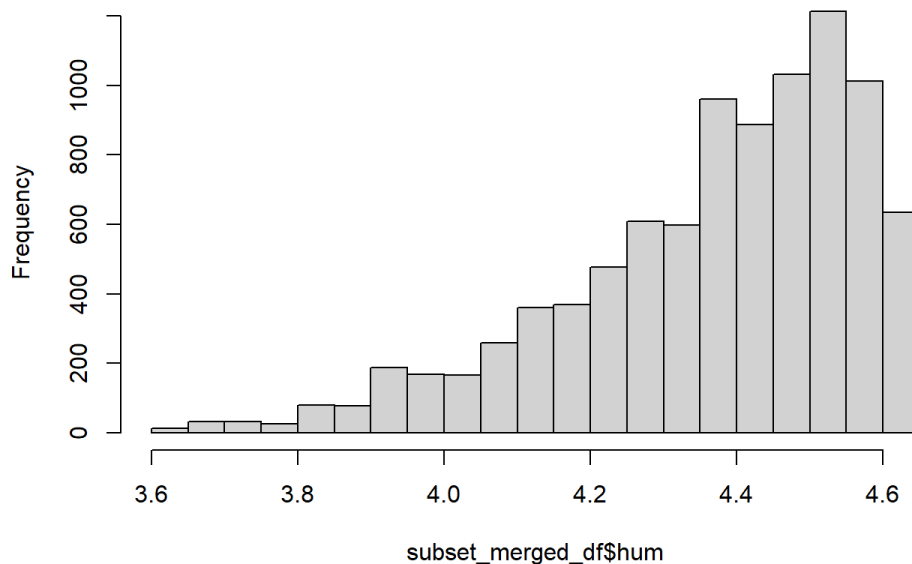
We can see that natural log is the best transformation method. Lets transform the values with natural log.

```
subset_merged_df$hum <- log(subset_merged_df$hum)
```

Lets take a look again after transforming hum

```
par(mfrow = c(1, 1))
hist(subset_merged_df$hum, main = "Distribution Plot for Humidity")
```

Distribution Plot for Humidity



Reflection on Challenges and Ease

- Challenges Faced
 - The most difficult challenge was finding a dataset that met the requirements. They have to be related, and you would also have to imagine how the tidied dataset would look like before determining if the datasets are suitable for the project.
 - Another challenge was figuring out the procedure for preprocessing the dataset, while trying to meet the required components and sections. In this project, we can see that the steps slightly deviated from the original/recommended template, but we still managed to address all preprocessing steps effectively.
- Ease and Strengths
 - The preprocessing steps were generally manageable. I was able to work with the dataset and code well with the guidance provided through lecture and practical lessons.

- By breaking down each task into smaller subtasks, I was able to organise the structure of the project despite the differences in steps from the template.
- Insights and Learning
 - As much as it was challenging, I learnt that visualising the preprocessing steps while only looking at the datasets is crucial in early decision making
 - I also realised that I was able to figure out the preprocessing methodology easier by reviewing lectures and practicing questions in both lecture and practical sessions. This helped me understand the conceptual knowledge better, and work out the preprocessing code easier (e.g. Scanning for Outliers)

References

1. Temperature and Humidity for Bats monitoring - Temperature and Humidity for Bats monitoring CSV - Victorian Government Data Directory. (2024). <https://discover.data.vic.gov.au/dataset/temperature-and-humidity-for-bats-monitoring/resource/c501ce88-bf15-5e6a-b41a-3e1a145091a3> (<https://discover.data.vic.gov.au/dataset/temperature-and-humidity-for-bats-monitoring/resource/c501ce88-bf15-5e6a-b41a-3e1a145091a3>)
2. Temperature and Humidity (Rising HF sensors) - Temperature and Humidity (Rising HF sensors) CSV - Victorian Government Data Directory. (2024). <https://discover.data.vic.gov.au/dataset/temperature-and-humidity-rising-hf-sensors/resource/14200bdb-fb16-51ba-a312-afac7f89a3d5> (<https://discover.data.vic.gov.au/dataset/temperature-and-humidity-rising-hf-sensors/resource/14200bdb-fb16-51ba-a312-afac7f89a3d5>)